Anyone with a Notion account can comment on this page. Sign up or login to comment

Domanda-risposte AIF

1. **Definizione di un ambiente o task, caratterizzazione. (Peas e task environment)**

   To define a rational agent we can define the environment in which it lives with the PEAS description. It stands for Performance measure, Environment type, Actuators Sensors. The performance measure is the measure that the agent aims to maximize. An example of PEAS description is this one for the taxi agent:

   - P: client satisfaction, destination reached, safety...
   - E: Streets, pedestrians, traffic
   - A: steering, breaks...
   - S: video, engine sensors, GPS

   The environment could be everything—the entire universe! In practice it is just that part of the universe whose state we care about when designing this agent—the part that affects what the agent perceives and that is affected by the agent's actions

   The environment type determines the type of agent we need, and can be simple or complex depending on these characteristics:

   - Observable: fully, partially, not
   - Single vs multi agent
   - Discrete vs continouos
   - Static vs dynamic
   - Episodic vs Sequential
   - Deterministic vs probabilistic

   The simplest form of environment is: Fully observable, determininstic, discrete, static, episodic, single agent. The table agent is the simplest one but also extremely unefficient, the others are: Simple reflex agent: selectactions on the basisof the current percept, ignoring the rest of the percept history. Model-based Reflex agents: keep track of the world it can't see now, by using an internal state. Goal based agent (problem solving): search and planning to find the action sequence that acheives the agent's goal. Utility-based agent: use an utility function, that is an internalization of the performance measure to distinguish between more and less desirable ways to get to the goal.

2. Ricerche informate/non:

   In the case of a problem solving agent, the action to take is not immediately claer and we need a plan to go ahead. The process to find the sequence of actions that forms a path to the goal is called search. This search can be:

   - Uninformed: The agent has no additional information about the problem beyond its definition and cannot estimate the distance to the goal. Some example of uniformed algorithms are the various three search algorithms like
     - BFS: the fringe is a FIFO queue
     - DFS: the fringe is a LIFO queue
     - Depth limited search: DFS with depth limit l, exclude solutions deeper than l
     - Iterative deepening search: start with low l, increase it.
     - Uniform cost search: expand least-cost unexpanded node, so order queue by cost. (equivalent tho BFS if all costs are equal)
   - Informed: The agent uses an heuristic function that provides an approximation of the distance to the goal for each node. This helps guide the search more efficiently by guiding the expansion choice.
     - Greedy best first: We use the value of the heuristic function as our distance to the goal, very dependant on how good the heuristic is.

- A* search: we avoid expanding nodes that are already too expansive, so our evaluation function is f(n) = g(n) + h(n) where g is the cost to reach n so far and h is the heuristic value for n. To work we need an admissible heuristic, meaning that the value of the heuristic never exides the true cost, for every node (h(n) ≤ h ∗ (n)).

Strategies are evaluated with this metrics: completeness (finds a solution if present), optimality(always finds the lowest cost solution), time and space complexity.

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening
Complete?	Yes[1]	Yes[1,2]	No	No	Yes[1]
Optimal cost?	Yes[3]	Yes	No	No	Yes[3]
Time	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$
Space	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(bm)$	$O(b\ell)$	$O(bd)$

1 complete if b (branching factor) is finite and so is the state space, 2 if action costs ≥ e > 0. 3 if action cost all indentical

3. Single state problem definition

   Defined by: initial state e.g. “Arad”, successor function which is a set of action-state pairs, goal test that implicitly contains the set of goal states and an additive path cost. A solution is a sequence of actions from the initial state to a goal state. Since the real world is too complex, we must abstract, so make assumptions that simplify the problem

4. Local Search algorithms

   There are some cases where we do not need the path to a solution, since the solution itself is the goal state. In such cases we can use local search algorithms that work by searching from a start state to its neighbors not keeping track of the paath or which states have been reached. They use little memory and can often find reasonable solutions to problems in large or infinte state spaces. Example: N-queens put on a board n*n board such that no queen attachs the others. Some examples are:

   - Hill climbing (greedy local search) where on each iteration we move to the neighboring state with higher value, meaning we go in the direction of the steepest ascent. It struggles with local minima and shoulders→ random restart and sideways moves.
   - Simulated Annealing: allow some bad moves with a probability (temperature) T and decrease the rate gradually.
   - Local Beam search: instead of keeping one state in memory, keep k and at each step keep the best k neighbors (or some randomly).
   - Genetic algorithms: Keep a number of states and at each timestep the best ones (highest value) are used to produce the successor states by combination.

5. Pseudocodice di hill-climbing

```
function Hill-Climbing(problem) returns a state that is a local maximum
   inputs: problem, a problem
   local variables: current, a node
                    neighbor, a node

   current ← Make-Node(Initial-State[problem])
   loop do
      neighbor ← a highest-valued successor of current
      if Value[neighbor] ≤ Value[current] then return State[current]
      current ← neighbor
   end
```

6. Pseudocodice algoritmo genetico

```
function GENETIC-ALGORITHM(population, fitness) returns an individual
  repeat
     weights ← WEIGHTED-BY(population, fitness)
     population2 ← empty list
```

```
    population2 ← empty list
    for i = 1 to SIZE(population) do
        parent1, parent2 ← WEIGHTED-RANDOM-CHOICES(population, weights, 2)
        child ← REPRODUCE(parent1, parent2)
        if (small random probability) then child ← MUTATE(child)
        add child to population2
    population ← population2
  until some individual is fit enough, or enough time has elapsed
  return the best individual in population, according to fitness

function REPRODUCE(parent1, parent2) returns an individual
  n ← LENGTH(parent1)
  c ← random number from 1 to n
  return APPEND(SUBSTRING(parent1, 1, c), SUBSTRING(parent2, c + 1, n))
```

Figure 4.8 A genetic algorithm. Within the function, *population* is an ordered list of individuals, *weights* is a list of corresponding fitness values for each individual, and *fitness* is a function to compute these values.

7. Pseudocodice Simuleted Annealing

   Idea: escape local maxima by allowing some bad news, gradually decreasing their frequency.

```
function Simulated-Annealing( problem, schedule) returns a solution state
   inputs: problem, a problem
           schedule, a mapping from time to "temperature"
   local variables: current, a node
                    next, a node
                    T, a "temperature" controlling prob. of downward steps

   current ← Make-Node(Initial-State [problem])
   for t ← 1 to ∞ do
       T ← schedule[t]
       if T = 0 then return current
       next ← a randomly selected successor of current
       ΔE ← Value[next] – Value[current]
       if ΔE > 0 then current ← next
       else current ← next only with probability e^(ΔE/T)
```

8. Continuous state spaces

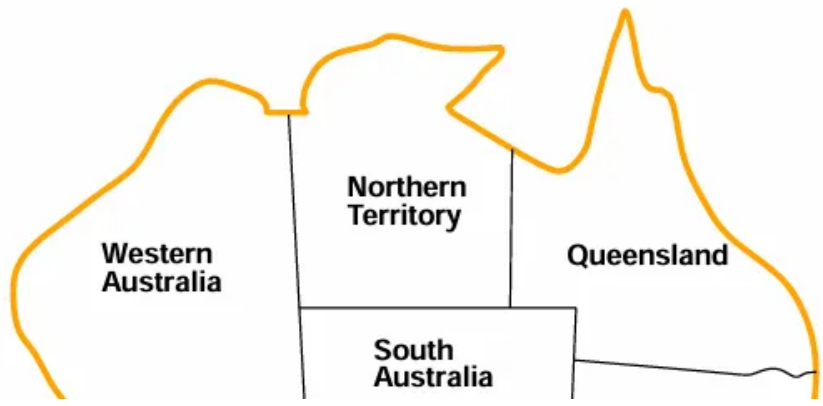
   to treat continuos state spaces one option is to discretize them , but that can be expansive. So we can have non-deterministic actions, so the agent doesn't know the state it is transitioned to after the action becoming a belief state of a contingency problem. An example is the erratic vacuum world, where sucking can lead both to a clean or dirty room. The solution, rather than sequences of actions are AND/OR trees called contingency plans.

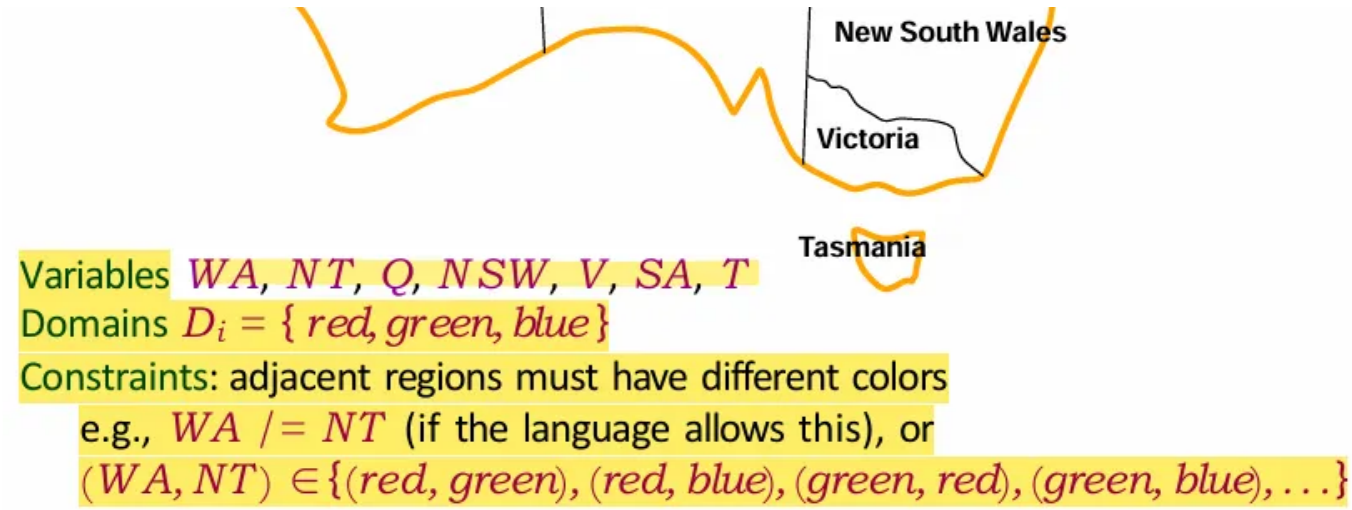
9. Descrivere cosa si intendendo con CSP, spiegando quando è utile applicare questa strategia, vantaggi e svantaggi e fornire un esempio -Definizione CSP con esempio

   A constraint satisfaction problem consist of 3 components:

   - X, a set of variables
   - D, a set of domains, one for each variable
   - C a set of constraints that specify allowable combination of values.

   An example is the map coloring problem, where solutions are assignments satisfying all constraints





A state is then defined as a set of variables Xi with values from domains Di and a solution is a complete assignement, meaning a state where each variable has an assigned value that is consisten with all constraints. A CSP can either have a binary or graph structure, where arcs show consistency between variables.

Backtracking search is the basic uninformed algorithm for solving CSPs. It uses Depth-first search with single-variable assignments. Backtracking search can be improved using heuristics:

- Minimum Remaining Values (MRV) : Choose the variable with the fewest values in its domain (also known as the Fail-first heuristic) and for ties choose the one with the most constraints on remaining variables. This helps to detect failures earlier.
- Least constraining value: Select the value that rules out the fewest values for other variables. This maximizes flexibility for future assignments.

With Forward checkhing instead we keep track of remaining legal values for unassigned variables and terminate the search when any variables has no legal values. This however does not ensure early detection for all failures, but we can do so by using arc consistency, which checks if a for every value of X there is some allowed value of Y in a X → Y constraint, by propagating constraints.

```
function AC-3( csp) returns the CSP, possibly with reduced domains
   inputs: csp, a binary CSP with variables {X1, X2, ..., Xn}
   local variables: queue, a queue of arcs, initially all the arcs in csp

   while queue is not empty do
      (Xi, Xj) ← Remove-First(queue)
      if Remove-Inconsistent-Values(Xi, Xj) then
         for each Xk in Neighbors[Xi] do
            add (Xk, Xi) to queue
```

```
function Remove-Inconsistent-Values( Xi, Xj) returns true iff succeeds
   removed ← false
   for each x in Domain[Xi] do
      if no value y in Domain[Xj] allows (x,y) to satisfy the constraint Xi ↔ Xj
         then delete x from Domain[Xi];  removed ← true
   return removed
```

Limitation is that they do not scale well.

10. Algoritmo Backtracking CSP

```
function Backtracking-Search(csp) returns solution/failure
   return Recursive-Backtracking({ }, csp)

function Recursive-Backtracking(assignment, csp) returns soln/failure
   if assignment is complete then return assignment
   var ← Select-Unassigned-Variable(Variables[csp], assignment, csp)
   for each value in Order-Domain-Values(var, assignment, csp) do
      if value is consistent with assignment given Constraints[csp] then
         add {var = value} to assignment
         result ← Recursive-Backtracking(assignment, csp)
```

```
            if result /= failure then return result
            remove {var = value} from assignment
    return failure
```

11. Pseudocodice Minimax - Pseudocodice $\alpha\beta$ pruning

```
function Minimax-Decision(state) returns an action
   inputs: state, current state in game
   return the a in Actions(state) maximizing Min-Value(Result(a, state))

function Max-Value(state) returns a utility value
   if Terminal-Test(state) then return Utility(state)
   v ← − ∞
   for a, s in Successors(state) do v ← Max(v, Min-Value(s))
   return v

function Min-Value(state) returns a utility value
   if Terminal-Test(state) then return Utility(state)
   v ← ∞
   for a, s in Successors(state) do v ← Min(v, Max-Value(s))
   return v
```

It is complete if the tree is finite, optimal against optimal opponent, $O(b^m)$ in time and $O(bm)$ in space. But we don't actually need to explore every path, and we can avoid it by using the $\alpha - \beta$ pruning, where α is the best value for MAX so anything below that he will ignore, and β the lowest for MIN, so anything above that is discarded. Very susceptible to problems of the heuristic functions, which determines the prune treshold.

```
function Alpha-Beta-Decision(state) returns an action
   return the a in Actions(state) maximizing Min-Value(Result(a, state))

function Max-Value(state, α, β) returns a utility value
   inputs: state, current state in game
           α, the value of the best alternative for  max along the path to state
           β, the value of the best alternative for  min along the path to state
   if Terminal-Test(state) then return Utility(state)
   v ← − ∞
   for a, s in Successors(state) do
      v ← Max(v, Min-Value(s, α, β))
      if v ≥ β then return v
      α ← Max(α, v)
   return v

function Min-Value(state, α, β) returns a utility value
   same as Max-Value but with roles of α, β reversed
```

12. Monte Carlo Tree search

The value of a state is estimated as the average utility over a number of simulations, so it uses NO heuristics. A playout (simulation that chooses moves untill terminal position is reached) is made of 4 phases:

- Selection: starts from the root and chooses a move by a selection policy
- Expansion: the search tree grows by generating a child of a selected node
- Simulation: playout from generated child node
- Backpropagation: use the result of the simulation to update all the search tree nodes going up to the root.

The selection policy is called Upper Confidence bounds applied to Trees (UCT)

$$UCB1(n) = \frac{U(n)}{N(n)} + C \times \sqrt{\frac{\log N(\text{PARENT}(n))}{N(n)}}$$

Where the first term is the exploitation term (average utility of n) and the second term (including the C) is the exploration term which favors nodes that have been explored few times.

13. Knowledge based agents

    A knowledge base is a set of sentences in a formal language and provide a declarative approach to building an agent since we Tell him what he needs to know. Then based on this knowledge and the current percept the agent decides the action.

    Knowledge base KB entails sentence α ↔ α is True in all worlds where KB is True. In particolar we say that m is a **model** of a sentence α if α is True in m, and KB entails α iff the models of α are a subset of the models of KB.

    KB ⊢ α means that α can be derived from KB with procedure i. The procedure is sound if whenever KB ⊢ α is also True that KB entails α, and is complete if the opposite is also true.

```
function KB-Agent(percept) returns an action
   static: KB, a knowledge base
           t, a counter, initially 0, indicating time
   Tell(KB, Make-Percept-Sentence(percept, t))
   action ← Ask(KB, Make-Action-Query(t))
   Tell(KB, Make-Action-Sentence(action, t))
   t ← t + 1
   return action
```

    An example is the Wumpus word and a sentence can be:

    $B_{1,1} \iff (P_{1,2} \vee P_{2,1})$ meaning that pits cause breeze in adjacent squares.

    The first approach to inference is by enumeration, but scales horribly and would require a depth first enumeration of all models.

    Another way is by logical equivalence, with the definition being that two sentences are logically equivalent iff they are true in the same models. α ≡ β iff α |=β and β |=α. We say that a sentence is valid if it is true in all models and we can use this to do inference by the deduction theorem "KB |=α if and only if (KB ⇒ α) is valid". A sentence is satisfiable if is true in some model, unsatisfiable if it is true in no model. Satisfiability is connected to inference via: KB |= α iff (KB ∧ ¬α) is unsatisfiable, so we prove by reductio ad absurdum.

14. Proof methods

    The goal is to find a chain of conclusions that lead to the goal. There are two kinds:

    - Application of inference rules: gives a sound way of generating new sentences from old ones, and the proof is a sequence of inference rule applications, requires normal form.
    - Model checking: it is a truth table enumeration improved by backtracking, sound but incomplete.

15. Resolution rule (Propositional logic)

    it is one of the possible proof methods used by logical agents.

$$\ell_1 \vee \cdots \vee \ell_k, \quad m_1 \vee \cdots \vee m_n$$
$$\ell_1 \vee \cdots \vee \ell_{i-1} \vee \ell_{i+1} \vee \cdots \vee \ell_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n$$

    where l_i and m_i are complementary literals e.g:

$$P_{1,3} \vee P_{2,2}, \quad \neg P_{2,2}$$
$$P_{1,3}$$

    Resolution is sound and complete for propositional logic. It requires that the KB is in a

Conjunctive normal form, meaning that each clause is a conjunction of disjunctions of literals (AND of ORs) E.g., (A ∨ ¬B) ∧ (B ∨ ¬C ∨ ¬D), so if that is not true, we must convert it. Then given the KB and a sentence α we want to prove, we apply the resolution rule to KB ∧ ¬α and if we derive the empty clause this means the assumption leads to a contradiction, so the original statement must be **entailed** by the knowledge base.

16. Forward and Backward chaining in Propositional logic

    The idea behind forward chaining is to fire any rule whose premises are satisfied in the KB and add its conclusions to the KB untill the query is found, may do lots of work that is irrelevant to the goal since it is data-driven. The backward chaining on the other hand, is goal driven, meaning we start from the query q and prove it by finding some rules that include it and prove those, untill we find a solution made of known facts. This avoids loops because we check if the subgoal is already on the stack, and avoids repeated work because we check if the subgoal has already been proven. Both these methods require that the KB and the query are written as a conjuction of Horn Clauses which can be a propositial symbol or a conjuction of symbols that imply a symbol e.g. C ∧ (B ⇒A) ∧ (C ∧ D ⇒B). The modus ponens for Horn forms is:

    $$\frac{a_1, \ldots, a_n, \qquad a_1 \wedge \cdots \wedge a_n \;\Rightarrow\; \beta}{\beta}$$

17. First Order Logic

    Propositional logic, though it has many pros, it also has the con of having very limited expressive power, for example to say that pits cause breeze in adjacent squares we must write a sentence per square. In FOL we assume that world contains:

    - Objects: people
    - Relations: brother of
    - Functions: Father of

    We also introduce quantifers:

    - Universal quantifier: ∀x P is True in a model m iff P is true with x being each possible object in the model. Must be followed by ⇒
    - Existential quantifier: ∃x P is True in a modelm iff P is true with x being some possible object in the model. Must be followed by ∧

    Each quantifier can be expressed as full negation of the other.

    You can go from FOL to proposition logic by existential and universal instantiation, which are a way to generalize on variables. For the first one it is sufficient to substitute the variable with a ground term, for the second one we introduce a new symbol called skolem constant. The UI can be applied many times to add sentences to the KB that stays logically equivalent, the EI can be applied once to replace the existential sentence and yealds a KB that is not equivalent to the old, but mantains the same satisfiability. Using these we can effectivly propositionalize the KB, but we generate lots of irrelevant sentences.

18. Unification

    Unify(α,β) = θif αθ= βθ. We can immediately get the inference if we can find a substitution such that $King(x)$ and $Greedy(y)$ match $King(John)$ and $Greedy(John)$ for example $\theta = \{x/John, y/John\}$ works. So basically the unify algorithm takes two sentences in input and returns a unifier for them (substitution) if one exists.

19. Generalized modus ponens (GMP) (con la formula, se vuoi anche con esempi)

    Generalized Modus Ponens is more general than Modus Ponens in the sense that the known facts and the premise of the implication need match only up to a substitution, rather than exactly.GMP is a sound inference rule that needs to be used with a KB of definite clauses (exactly one positive literal)

    $$\frac{p'_1,\ p'_2,\ \ldots,\ p'_n, \qquad (p_1 \wedge p_2 \wedge \ldots \wedge p_n \Rightarrow q)}{q\theta}$$

    where $SUBST(\theta, p'_i) = SUBST(\theta, p'_i)$ for all i

    Example

Example:

$$King(John), Greedy(John) \quad King(x) \wedge Greedy(x) \Rightarrow Evil(x))$$
$$Evil(John)$$

where $\theta = \{x/John\}$

20. Psuedo codice Foward chaining del First Order Logic

```
function FOL-FC-Ask(KB,a) returns a substitution or false
  repeat until new is empty
      new ← { }
      for each sentence r in KB do
          (p1 ∧ ... ∧ pn ⇒ q) ← Standardize-Apart(r)
          for each θ such that (p1 ∧ ... ∧ pn)θ = (p'1 ∧ ... ∧ p'n)θ
                      for some p'1,...,p'n in KB
              q' ← Subst(θ,q)        the only thing that changes between the two is the variables
              if q' is not a renaming of a sentence already in KB or new then do
                  add q' to new
                  φ ← Unify(q',a)
                  if φ is not fail then return φ
      add new to KB
  return false
```

Sound and complete for FOL, since every inference is the application of GMP which is sound. May not terminate if α is not entailed.

21. Backward chaining algorithm

```
function FOL-BC-Ask(KB,goals,θ) returns a set of substitutions
  inputs: KB, a knowledge base
          goals, a list of conjuncts forming a query (θ already applied)
          θ, the current substitution, initially the empty substitution { }
  local variables: answers, a set of substitutions, initially empty

  if goals is empty then return {θ}
  q' ← Subst(θ,First(goals))
  for each sentence r in KB   first find a sentence in KB whose tail coincides with the goal
          where Standardize-Apart(r) = (p1 ∧ ... ∧ pn ⇒ q)
          and θ' ← Unify(q,q') succeeds   means substituting the variable of the clause with the one in the goal
      new_goals ← [p1,...,pn|Rest(goals)]
      answers ← FOL-BC-Ask(KB,new_goals,Compose(θ',θ)) ∪ answers
  return answers
```

incomplete due to infinite loops and space linear in size of proof.

22. Spiegare PROLOG e fare un esempio

Prolog is a logic programming language that uses backward chaining with horn clauses to derive the answer from a set of clauses. A problem is a set of clauses $head$: $-literal_1, ..., literal_n$. It uses the closed world assumption, meaning everything that is not known to be True is False. An example is:

```
append([], Y, Y). %the base case appending Y to an empty list yealds Y
append([X|L], Y, [X|Z]) :- append(L,Y,Z). %To append [X|L] and Y to get [X|Z],
you must append L and Y to get Z query: append(A,B,[1,2]) ? answers: A=[]
B=[1,2] or A=[1] B=[2] or A=[1,2] B=[]
```

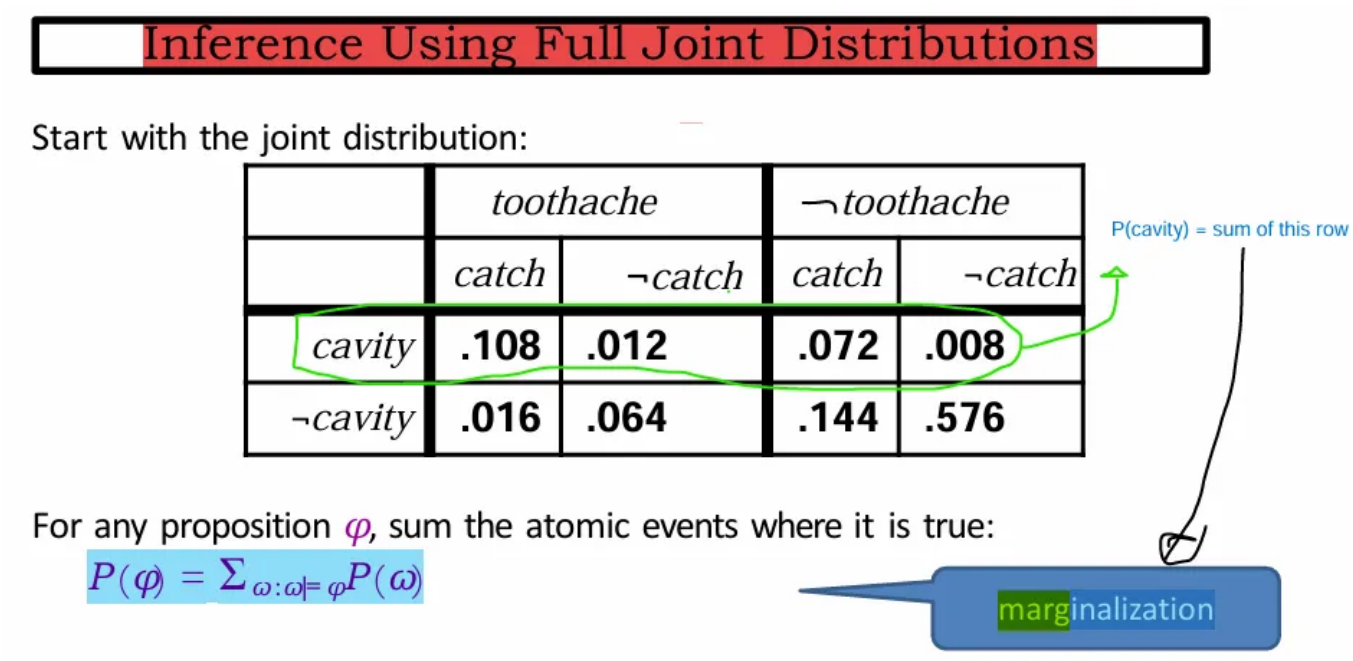
23. Definizione PDDL, con esempio e action schema

PDDL is a formal language for planning that extends classical planning by allowing agents to deal with more complex environments such as partially-observable, non deterministic,… A state in PDDL is defined as a conjuction of atomic ground fluents and a problem consists of an

initial state, one or more goal states, and a set of action schemas characterized by a name, a precondition and an effect. An action can be applied if its preconditions can be unified with the fluents of the current state and its result produces a state in which we add the positive literals in the effect and remove the negative literals in the action's effect.

```
Action(Fly(p, from, to)), PRECOND: At(p, from) ∧ Plane(p) ∧ Airport(from) ∧
Airport(to) EFFECT: ¬At(p, from) ∧ At(p, to) %we are no longer in from so we
remove it
```

24. Marginalization with full joint distribution



Where for example $P(catch) = \sum_{\omega:\omega=catch} P(\omega)$, so we only sum the atomic events where catch (ϕ) is true.

25. Scrivere quello che ricordate su naive bayes models.

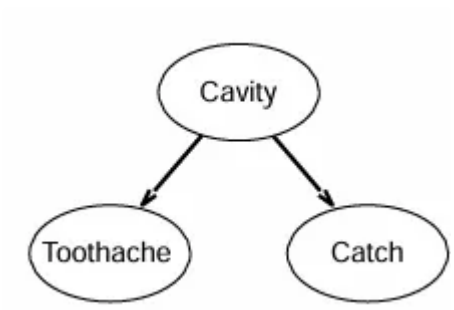
Naive bayes model are the application od the Bayes rule

$$P(cause/effect) = \frac{P(effect/cause)P(cause)}{P(effect)}$$

In these models the full joint distribution can be written with a number of parameters linear in n as:

$$P(cause, effect_1, ..., effect_n) = P(cause) \prod_i P(effect_i|cause)$$

The model is called naive because it makes the strong assumption that all effects are conditionally independent given the cause, but is still usefull in practice.

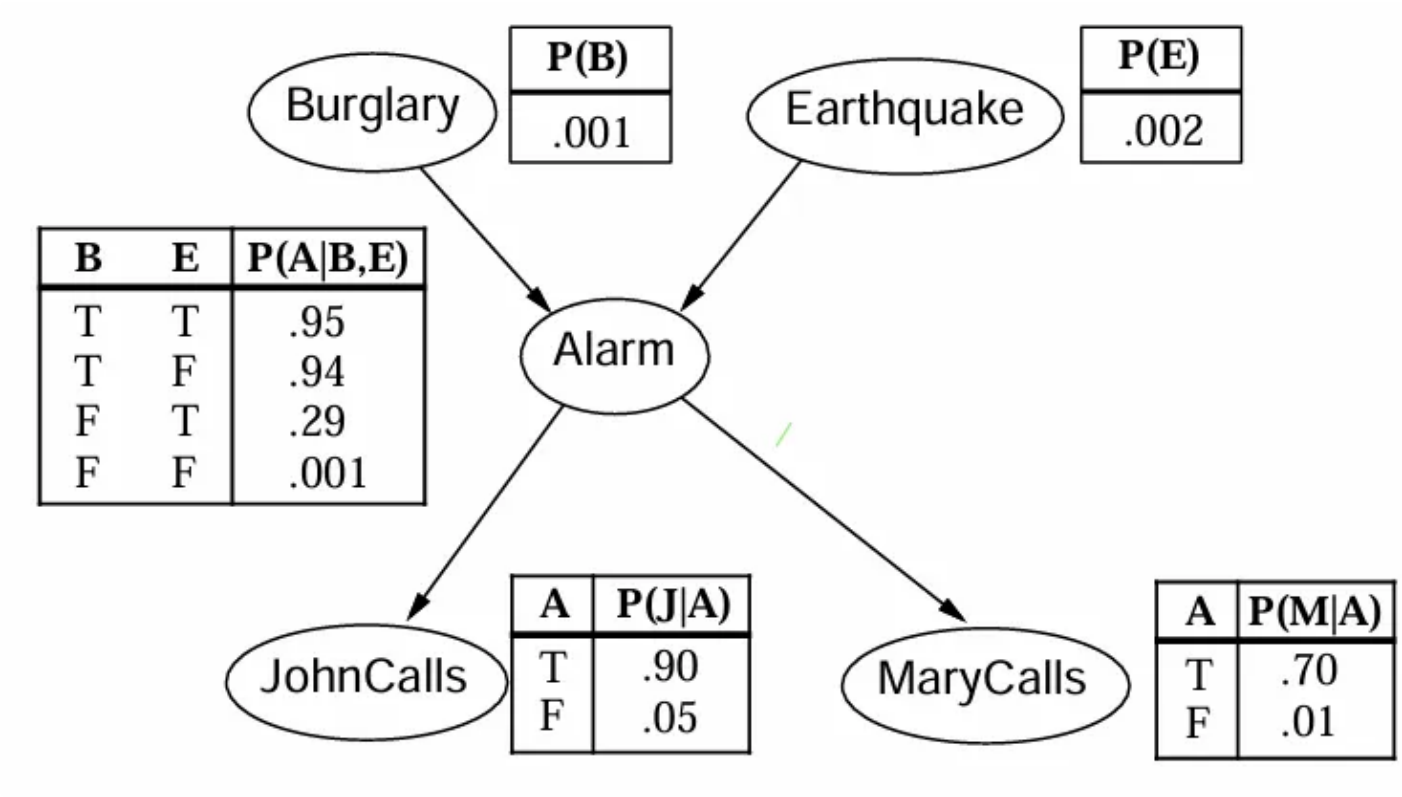


26. Parla delle Bayesian Net (se vuoi anche con esempi)

In bayesian networks we represent dependencies among variables throu a directed acyclic graph. It is made of a set of nodes, one per variable, and the edges represent dependencies. For each node we a conditional distribution given its parents $P(X_i|Parents(X_i)$. We can see the model at two levels:

- Local semantics: tell us that each node is conditionally independent of it nondescendants given its parents
- Global semantics: define the full joint distribution: $P(x_1, ...x_n) = \prod_{i=1}^{n} P(x_i|parents(X_i))$

Moreover we have that each node is conditionally independent of all others given its Markov Blanket: parents + children + children's parents.

Blanket: parents + children + children's parents.

When constructing a Bayesian network the ordering of the variables is important.